RapidBoxForest: Link-Envelope-Guided Box Forests for Fast Multi-Query Manipulation

TIAN, Yu and Ren, Hongliang

***Abstract*—Repeated-query manipulation needs reusable free-space structure, yet constructing high-quality convex regions can be expensive in high-dimensional scenes and fragile under local edits. This paper presents RapidBoxForest (RBF), a link-envelope-guided box-forest planner for this low-build-cost regime. RBF repurposes the classical endpoint-bounding pipeline from swept-capsule collision detection as a configuration-space box-validation oracle: endpoint interval envelopes over a joint box are radius-lifted into link envelopes, which validate volumetric regions rather than individual path segments. Accepted boxes are grown into a scene-specific forest, consolidated into a typed corridor graph, and queried by membership lookup plus graph search. A Lifelong Envelope Cache Tree (LECT) separately stores kinematics-keyed endpoint and link-envelope evidence, while the scene-level RBF cache stores validated boxes and connectivity for one environment. The resulting planner trades regional expressivity for inexpensive box validation, simple maintenance, and explicit reuse boundaries. Its formal claim is soundness for paths contained in conservatively validated box unions; filters and post-processing are accepted through the configured query-validation path. Experiments study repeated Shelf+IIWA planning, envelope representations, evidence replay, and local obstacle updates.**

***Index Terms*—Motion planning, multi-query planning, link interval envelopes, box-region planning, configuration-space planning.**

I. Introduction

High-DOF manipulators often face many related queries in nearly fixed workcells: the robot kinematics persist, while start-goal pairs, local obstacles, or small scene edits change. Reuse is valuable only if the reusable object is cheap enough to build, update, and query.

Convex-region methods such as IRIS and Graphs of Convex Sets provide powerful region-level planning abstractions once certified regions are available [1], [2], [3], but their construction cost can dominate in high-dimensional or frequently edited scenes. Sampling-based methods such as PRM and RRT variants avoid heavy preprocessing and solve single queries effectively [4], [5], [6], [7], yet their reusable structures are usually zero-volume roadmaps or trees.

RBF targets the middle ground: it grows axis-aligned C-space boxes, validates them with a link-envelope scene oracle, connects accepted boxes into a graph, and answers later queries by searching the resulting corridor. The design trades region expressivity and path optimality for low build cost, simple adjacency, and explicit multi-query reuse. The resulting planning object is intended for regimes in which repeated-query latency and rebuild cost matter more than obtaining a tighter regional decomposition.

a) Problem statement.: Let $\mathcal{Q} \subset \mathbb{R}^n$ be the joint-limit box of a fixed manipulator and let $\mathcal{O} \subset \mathbb{R}^3$ be the current workspace obstacle set. Given a batch or stream of start-goal queries $(q_s, q_g) \in \mathcal{Q}^2$, the objective is to build a scene-specific planning object that supports repeated query retrieval while amortizing geometric work that depends only on robot kinematics and joint intervals. The object sought by RBF is a finite family of validated C-space boxes and an adjacency graph over those boxes. Its construction should have lower rebuild and local-update cost than a high-quality convex-region decomposition, while still exposing volumetric free-space structure rather than only path-local collision checks.

The key geometric ingredient is not a new capsule swept-volume identity, but a new use of the classical endpoint-bounding pipeline. For rounded link models, RBF separates the swept capsule into the sweep of a zero-radius center segment and a final radius lift. Endpoint interval envelopes bound the two endpoint trajectories over a joint box; the convex hull of these endpoint envelopes, enlarged by the link radius, encloses the swept rounded link. RBF uses this bound as a C-space box free-space validation oracle. The same endpoint evidence can be exported as axis-aligned bounding boxes (AABBs), SupportHull records, or staged AABB–SupportHull filters. Conservative endpoint sources yield a conditional conservative box certificate, whereas tighter practical sources act as planning filters under the common query-validation policy.

The planning layer builds directly on this primitive. A sample-guided grower selects query-relevant frontiers, materializes local candidate boxes around projected seeds, and extends a connected forest of validated regions. This is a workload-biased adaptive cell decomposition in manipulator C-space. RBF has two reuse levels. The scene-level RBF cache is the constructed box forest and corridor graph for a fixed obstacle set; it directly answers many start-goal queries in that environment. The lifelong envelope cache, LECT, memoizes scene-independent endpoint

The authors are with the Department of Electronic Engineering, The Chinese University of Hong Kong (CUHK), Hong Kong SAR, China. E-mail: ty1997@link.cuhk.edu.hk.

TABLE I
Qualitative comparison of reusable manipulation-planning paradigms.

Family	Reuse object	Strength	Limitation	Relation to RBF
PRM / LazyPRM	Roadmap	Multi-query graph	Edge checks; zero volume	RBF reuses validated boxes rather than only edges.
RRTConnect / RRT	Query tree	Fast discovery	Little persistent reuse	RBF turns frontier growth into reusable box growth.
BIT* / FMT*	Sample graph	Anytime improvement	Query-centric checking	RBF prioritizes reusable low-cost regions.
IRIS / GCS	Convex regions	Optimization-ready graph	Costly inflation	RBF uses cheaper boxes instead of rich convex sets.
RBF	Box forest + LECT	Volume reuse	Conservative boxes	CCD-style link envelopes validate C-space boxes; scene and kinematic caches remain separate.

and link-envelope evidence keyed by robot kinematics and joint intervals; it accelerates future builds or local updates, while free-space labels remain scene-dependent.

The evaluation is organized around this design point. It examines repeated-query Shelf+IIWA planning, evidence replay, link-envelope representation trade-offs across AABB, SupportHull, and staged cascades, and local maintenance under obstacle edits. The resulting claims are framed around build cost, multi-query reuse, envelope-representation trade-offs, and update behavior rather than around a universal ranking against all single-query planners.

RBF is therefore a planning-systems contribution enabled by a geometric primitive. The endpoint-to-link envelope supplies the C-space box-validation oracle; the scene-level box-forest cache, lifelong LECT evidence cache, and local update rules show how that oracle can be assembled into a reusable multi-query planning pipeline.

The paper contributes RBF**, a link-envelope-guided box-region planner that lifts classical swept-capsule endpoint bounds from collision detection to reusable C-space free-space construction.** Its main algorithmic components are:

1) *C-space box validation from endpoint-to-link envelopes.* RBF repurposes CCD-style endpoint bounds as a joint-box free-space oracle, with conservative sources for certificates and tighter staged sources for query-validated filtering (section III).
2) *A reusable box-forest planner.* Seed descent, leaf-sweep coverage, restricted refinement, frontier expansion, and consolidation produce a scene-level typed corridor graph of validated boxes (section V).
3) *Two-level cache reuse.* Scene RBF caches store the current environment's box forest, while LECT stores lifelong kinematics-keyed endpoint and link-envelope evidence reusable across builds (sections IV and V).
4) *Local maintenance under scene edits.* Deletions revalidate formerly blocked domains for promotion, and insertions invalidate conflicting boxes before bounded local refill (section V-G).

The rest of the paper develops the envelope oracle (section III), LECT reuse layer (section IV), box forest and update rules (section V), and evaluation (section VI).

II. Related Work

A. Convex Region Planning and Graphs of Convex Sets

Convex region planners approach motion planning by constructing a certified cover of free space and then solving graph search or convex optimization on top of that cover. IRIS introduced the now-standard idea of computing large collision-free convex regions by alternating separating hyperplanes and region inflation [1]. Later work extended the same line into configuration space, improved region quality, or improved graph compactness through visibility-graph and clique-cover constructions [8], [9], [10]. Marcucci *et al.* made the Graph-of-Convex-Sets formulation particularly influential by showing that motion planning can be cast as optimization over a graph of certified convex regions [2], [3].

RBF shares the region-level objective of this literature but uses a different primitive. Its regions are axis-aligned boxes in C-space rather than general polytopes, so the cover is less expressive than a high-quality convex decomposition. The payoff is a lower-cost validation loop, simple adjacency bookkeeping, and direct local maintenance. The relevant comparison is therefore not whether boxes are intrinsically better than polytopes; it is whether a lower-cost region primitive can support repeated-query planning before the cost of tighter convexification is justified.

B. Swept Volume Approximation and Capsule Geometry

Computing the swept volume of a moving rigid body has been studied extensively in computational geometry and computer-aided manufacturing (CAM) communities [11], [12]. For a convex body $P = \mathrm{conv}(v_1, \ldots, v_m)$, bounding each generator trajectory by a conservative outer set $\widehat{\Gamma}_i(I)$ yields

$$\mathcal{S}_I(P) \subseteq \mathrm{conv}\Big(\bigcup_i \widehat{\Gamma}_i(I)\Big),$$

a bound that is implicit in oriented bounding box (OBB) tree construction for swept mesh primitives [13].

For capsule links this bound reduces to bounding only the two segment endpoints. The Proximity Query Package (PQP) exploits exactly this Minkowski decomposition ($C = S \oplus B_2(r)$, sweep of C reduces to sweep of S plus radius lift) [14]; Gilbert–Johnson–Keerthi (GJK) and its robust implementation by van den Bergen apply the same

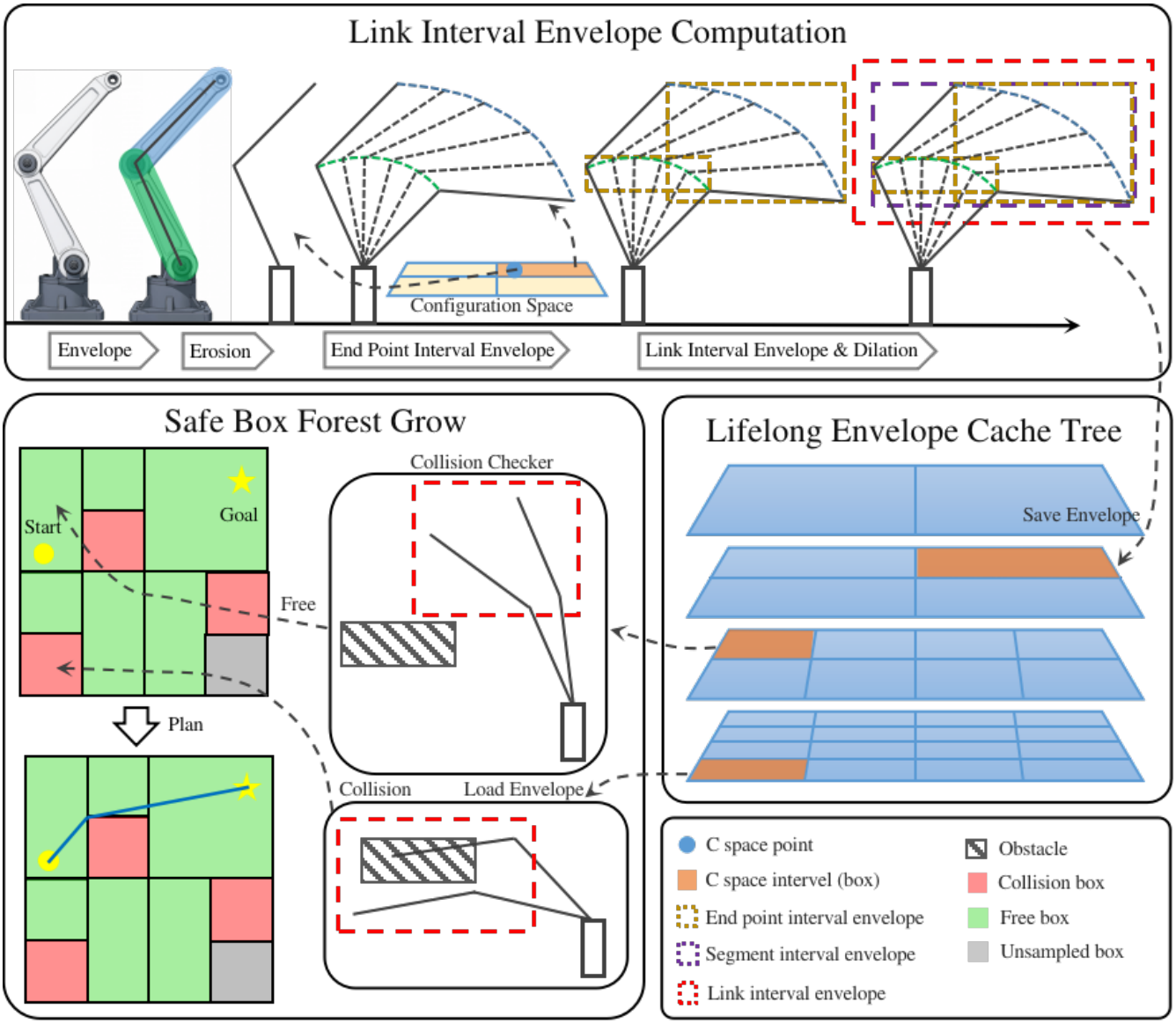


Fig. 1. RBF pipeline for link-envelope-guided multi-query planning. Top: endpoint interval envelopes induce link interval envelopes over a C-space box. Middle: LECT reuses kinematics-keyed endpoint and link-envelope records across repeated builds. Bottom: scene filtering and sample-guided growth expand a connected scene-level box forest for repeated corridor queries.

identity for distance queries [15], [16]; the Flexible Collision Library (FCL) generalises the swept-sphere interface to a full collision library [17].

C. Interval Forward Kinematics and the CCD Endpoint-Bounding Pipeline

Interval arithmetic [18], [19] provides inclusion-monotone enclosures of real-valued functions over parameter intervals. Snyder applied interval arithmetic to parametric geometry to compute conservative bounding boxes for curves and surfaces without mesh sampling [20]. Merlet extended DH-chain interval propagation to certified workspace analysis for parallel and serial manipulators [21], [22].

These ideas meet directly in the continuous collision detection (CCD) literature for articulated bodies. Redon, Kim, Lin, and Manocha [23] model each robot limb as a line-swept sphere (LSS, i.e. capsule) and propagate interval Denavit-Hartenberg (DH) transforms along the kinematic chain to obtain *interval vectors bounding the two LSS endpoint positions* over a motion time step. Algebraically, this is the radius-isolation identity $C = S \oplus B_2(r)$ followed by endpoint bounding of the zero-radius segment S and a final radius lift. The resulting swept capsule is bounded by padding the endpoint bounding box with the link radius. Thus CCD already uses the two-step pipeline

$$\textit{interval FK endpoint bounding} \longrightarrow \mathrm{bbox}([\mathbf{p}_i] \cup [\mathbf{p}_{i+1}]) \oplus B_2(r) = \textit{link swept-capsule bound}$$

which RBF reuses with a different parameter domain and planning role. The journal extension by Redon, Lin, Manocha, and Kim [24] applies the same framework to general robot arms and kinematic trees, making it a representative reference point for articulated-body CCD. Zhang *et al.* [25] replace interval arithmetic with Taylor models

in the same pipeline to reduce wrapping errors, analogous to how RBF's HIFK source tightens the unsplit IFK bound by recursively subdividing joint boxes. Schwarzer *et al.* [26] address a multi-query motivation similar to RBF's—reusing collision evidence across planning calls—but at the path-segment level rather than as volumetric C-space records. Corrales *et al.* [27] apply sphere-swept line (SSL) bounding volumes dynamically for human–robot safety monitoring using the same structural observation that bounding a capsule's motion reduces to bounding its two endpoints.

D. Novelty Boundary: Prior CCD vs. RBF

The two-step endpoint-interval-to-swept-capsule pipeline, including radius isolation and final radius lift, is therefore already present in the CCD literature for time-parameterized motion. The contribution boundary in this paper is the use of that pipeline as a C-space free-region-construction primitive. The differences are: (a) the parameter domain is a *C-space joint-box interval* rather than a trajectory time step, so the goal is to validate a volumetric free-space region rather than to test a single path segment; (b) the endpoint source is a switchable certified interface—IFK_AA and HIFK at different subdivision depths—rather than a fixed algorithm, allowing the tightness vs. cost trade-off to be controlled without changing the link-representation or planner layers; (c) compact per-representation link records (AABB and SupportHull) are organized for staged filtering during box growth, a design dimension absent from CCD libraries; (d) accepted boxes are assembled into a typed corridor graph whose overlap edges, tolerance-gap candidates, and segment witnesses have different certificate semantics; and (e) the endpoint and link-envelope records are organized in LECT, a persistent *kinematics-keyed* cache that reuses evidence across multi-query builds in related scenes, a reuse level not addressed by CCD libraries or path-segment caches. This distinction fixes the novelty boundary used throughout the paper. CCD uses the endpoint-bounding pipeline to enclose the swept volume of a body over a given time interval. RBF uses the same geometric pipeline over a joint box to validate candidate volumetric free-space regions, stores compatible evidence for replay, and organizes accepted regions into a queryable planning graph. Link envelopes therefore reduce repeated geometric work during free-region construction, while the query validation remains the acceptance rule for post-processed paths.

E. Multi-Query Reuse and Adaptive Configuration-Space Decomposition

Multi-query planners typically amortize effort by reusing a graph structure such as a roadmap. PRM is the classical example: build a connectivity graph once and reuse it for many start-goal pairs [6], [28]. LazyPRM, lazy edge checking, and certificate-based planners postpone or cache edge validation so that online search does not repeatedly evaluate the same discrete local connections [29], [30]. Experience-based and trajectory-library systems reuse prior solutions or local motion snippets, often as initialization or bias for a new query rather than as a conservative decomposition of a region of configuration space [31].

RBF separates reuse into two levels. Within a fixed scene, the constructed box forest and corridor graph are the reusable planning object, so later queries reuse volumetric boxes rather than only paths, binary collision outcomes, or zero-volume edges. Across related builds or local scene edits, LECT can replay kinematics-keyed envelope evidence while the scene-dependent free-space decision is recomputed. PRM-like methods reuse connectivity and must revalidate affected edges after scene changes. Convex-region planners reuse higher-quality regions, but those regions are usually coupled to the obstacle layout in which they were constructed. RBF makes the boundary explicit: scene-level boxes are reused inside one environment, while kinematic envelope records can outlive that environment.

The method also inherits the central idea of adaptive cell decomposition and subdivision planning: represent free space by cells, split cells where the current evidence is insufficient, and search the adjacency graph induced by the accepted cells [32], [33]. The difference is the operating regime. Classical cell-decomposition examples are often low-dimensional workspace or explicit C-space constructions, whereas RBF applies the same adaptive-cell idea to high-dimensional manipulator C-space through a link-envelope oracle. Scene-specific obstacle filtering is layered on top of reusable kinematics-keyed envelope evidence, with LECT available only as a cache for expensive records and the scene-level forest serving as the direct multi-query cache. The trade-off is explicit: RBF accepts axis-aligned boxes and over-approximation in exchange for simple validation, adjacency maintenance, and repeated-query throughput.

F. Sampling-Based Planning and Path Refinement

Sampling-based planners remain standard baselines when the goal is to solve a single query quickly. RRT-Connect is particularly effective in manipulator planning because bidirectional growth often succeeds with very little global structure [34]. Asymptotically optimal and informed variants such as RRT*, Batch Informed Trees (BIT*), and Fast Marching Tree (FMT*) improve path quality and search efficiency when more time is available [7], [35], [36], [37]. The Open Motion Planning Library (OMPL) has made these methods easy to deploy and benchmark in a common software stack [38], [39]. However, standard RRT variants produce single-use zero-volume evidence and typically restart from scratch on the next query.

By contrast, RBF uses sample-guided, frontier-biased expansion to materialize C-space boxes from the envelope oracle, producing a volumetric corridor structure that can

be queried repeatedly within the same environment. Trajectory optimizers address a different stage of the pipeline. CHOMP, STOMP, TrajOpt, and Gaussian-process motion planners refine an initial path by local descent or sequential convex optimization [40], [41], [42], [43]. RBF is complementary to these methods: it supplies a reusable region graph and leaves local smoothing, repair, and path checking to the downstream query pipeline.

Table I summarizes the resulting design trade-offs. The comparison is qualitative: it identifies what type of object each family reuses and where RBF fits in the chain from geometric evidence to C-space free-region planning, rather than ranking methods by the experimental results reported later.

III. Link Interval Envelopes

This section defines the geometric primitive used by RBF: a reusable map from a joint box to compact link interval envelopes. The goal is not exact swept volume computation, but a low-cost outer representation for box growth. Strict endpoint sources give conservative certificates; tighter sampled or support-based sources act as filters before query validation.

Let $\mathcal{Q}$ be the joint-limit box, $I \subseteq \mathcal{Q}$ a joint interval, and $\mathcal{O} \subset \mathbb{R}^3$ the obstacle set. Endpoint envelope $E_k(I)$ outer-bounds endpoint k, and link envelope $B_\ell(I)$ outer-bounds link ℓ over I, yielding the primitive $I \mapsto \{E_k(I)\} \mapsto B_\ell(I)$. Conservative endpoint evidence gives a conservative link envelope; empirical evidence is used only as a filter.

Assumption 1 (Collision model for certificates)**.** *The formal certificates in this paper concern collision between the configured active rounded links and the current workspace obstacle set $\mathcal{O}$. Joint limits are enforced by $I \subseteq \mathcal{Q}$. Self-collision constraints, attached objects, or task-specific forbidden regions can be included by adding the corresponding link-obstacle or link-link checks to the same scene-validation policy; if they are not enabled, they are outside the certificate stated below.*

A. Geometric Foundation

Let $T(q)x = R(q)x + t(q)$ denote the rigid transform induced by a configuration $q \in I$. For a reference body $C_0 \subset \mathbb{R}^3$, its pose at q is $C(q) = T(q)C_0$. We write $\mathcal{S}_I(C_0)$ for the exact workspace set swept as q ranges over the joint box I and $\mathcal{C}_I(C_0)$ for its convex envelope. RBF usually works with the convex envelope because the downstream representations are AABBs or support-function hull records. The construction below deliberately reuses two classical facts—capsule Minkowski decomposition and convex-hull generator bounding—that already appear in swept-sphere collision libraries and articulated-body CCD work [23], [24]. The change is the domain and downstream object: the parameter set is a C-space joint box, and the output is a candidate box-region for a planning graph.

Radius isolation. For a capsule $C = S \oplus B_2(r)$, rigid motion commutes with the fixed Euclidean ball, giving exact two-stage sweep bounds:

$$\mathcal{S}_I(C) = \mathcal{S}_I(S) \oplus B_2(r), \qquad \mathcal{C}_I(C) = \mathcal{C}_I(S) \oplus B_2(r).$$

RBF therefore follows the same centerline-first, radius-lift construction used by swept-sphere collision methods. This identity is exact for capsules; for sharp polytopes it requires the shape to be a rounded offset body.

Convex-hull generator bound. For a convex body $P = \mathrm{conv}(v_1, \dots, v_m)$, bounding each vertex trajectory by an outer set $\widehat{\Gamma}_i(I)$ suffices to bound the swept polytope:

$$\mathcal{S}_I(P) \subseteq \mathrm{conv}\Big(\bigcup_i \widehat{\Gamma}_i(I)\Big).$$

For a capsule link the seed is the center segment $[a, b]$, so only two endpoint trajectories need to be bounded.

Combining these two facts yields the practical recipe. For a capsule link $[a, b] \oplus B_2(r)$, RBF bounds only the two endpoint trajectories over the joint box, forms their convex hull, and applies the radius lift:

$$\widehat{\mathcal{C}}_I([a, b] \oplus B_2(r)) = \mathrm{conv}\big(\widehat{\Gamma}_a(I) \cup \widehat{\Gamma}_b(I)\big) \oplus B_2(r).$$

The numerical task reduces to computing reusable envelopes for two endpoint trajectories per link over I. Those records are then used by the planner to classify I as a candidate box-region, not merely to answer a path-segment CCD query.

Proposition 1 (Endpoint envelopes induce link interval envelopes)**.** *Consider a rounded link $L = [a, b] \oplus B_2(r)$ and a joint interval I. If endpoint envelopes $E_a(I)$ and $E_b(I)$ satisfy $\Gamma_a(I) \subseteq E_a(I)$ and $\Gamma_b(I) \subseteq E_b(I)$, then*

$$\mathcal{S}_I(L) \subseteq \mathrm{conv}\big(E_a(I) \cup E_b(I)\big) \oplus B_2(r).$$

Consequently, any outer representation of the right-hand side is a valid link interval envelope for the whole joint box I.

Proof. For each $q \in I$, the center segment of the moved link is the convex hull of the two moved endpoints, so $T(q)[a, b] \subseteq \mathrm{conv}(\Gamma_a(I) \cup \Gamma_b(I))$ over the entire interval. The endpoint inclusions place this set inside $\mathrm{conv}(E_a(I) \cup E_b(I))$. Adding the fixed radius ball commutes with the rigid-motion sweep of the capsule, giving the stated containment. □

Corollary 1 (Conservative joint-box validation)**.** *Let $I \subseteq \mathcal{Q}$ be a joint box and let $\{B_\ell(I)\}$ be conservative link interval envelopes for all active rounded links. If $B_\ell(I) \cap \mathcal{O} = \emptyset$ for every link ℓ, then every configuration $q \in I$ is collision-free with respect to the collision model in assumption 1.*

Proof. By proposition 1, each conservative link envelope contains the swept workspace volume of its link over all configurations in I. Disjointness from $\mathcal{O}$ for every active link therefore excludes workspace obstacle collision for the whole manipulator at every $q \in I$ under assumption 1. □

B. Endpoint Interval Envelopes

Figure 1 illustrates the AABB extraction pipeline. Endpoint interval axis-aligned bounding boxes (iAABBs) are

computed by interval forward kinematics (IFK) or tighter certified AA-based variants, combined into a conservative zero-radius centerline envelope, and padded by the link radius before collision querying. The remaining numerical task is to outer-bound each endpoint trajectory over I by a reusable set $E_k(I)$. Endpoint envelopes are the common interface between kinematic propagation and the downstream link-envelope representations.

For endpoint k, let $\Gamma_k(I)$ denote its exact workspace trajectory over interval I. Any conservative set that contains $\Gamma_k(I)$ is an endpoint interval envelope. The basic endpoint record stores this envelope as an axis-aligned bounding box, because boxes are efficient to store, easy to combine, and directly reusable by all downstream link representations.

a) Interval forward kinematics (IFK).: The IFK endpoint source used by the planner is backed by affine-arithmetic forward kinematics. It propagates workspace bounds along the DH chain and provides the lowest-cost certified member of the endpoint-source family. Its limitation is the usual wrapping effect: on wide boxes or long chains, one evaluation accumulates residual over-approximation across the full joint box.

b) Hierarchical IFK (HIFK).: HIFK further reduces wrapping by recursively bisecting the joint box and evaluating AA-FK at the leaves before hulling the child endpoint boxes. HIFK is therefore a distinct endpoint source rather than a naming variant of unsplit IFK: a depth-1 HIFK evaluation already splits once and hulls two leaves, whereas IFK performs one unsplit AA-FK pass on the original box. The split depth is an endpoint-source parameter: larger depths reduce wrapping at the cost of additional FK evaluations.

c) Incremental FK state reuse.: Tree descent also carries a transient FK state: interval-valued prefix transforms $[\boldsymbol{T}^{0:0}], \ldots, [\boldsymbol{T}^{0:n}]$ and the per-joint DH interval matrices used to build them. The full prefix chain is computed once at the root or after a restart; when only I_k changes, prefixes before k remain valid and only the suffix is recomputed. This short-lived state is not the persistent cache itself; it only keeps endpoint materialization efficient before LECT decides whether to retain the evidence.

Taken together, these sources define a clear certified spectrum: IFK_AA is the lowest-cost source, while increasing HIFK depth offers progressively tighter outer bounds at increasing cost. Endpoint envelopes are the right intermediate abstraction for the planner because they isolate the kinematic bounding problem from the scene-query and graph-building problems. The downstream link-envelope layer remains fixed while the upstream source changes with the certification and runtime budget, and LECT can cache the resulting evidence without depending on the obstacle set.

C. Link-Envelope Representations

Once endpoint envelopes are available, the remaining design question is how to represent the swept link volume compactly for box growth, collision filtering, and optional reuse. For a capsule link, RBF does not reason about the thick link directly. It first bounds the zero-radius center segment between the two endpoint envelopes and then pads that centerline envelope by the link radius. A conservative link interval envelope $B_\ell(I)$ is any set that contains the full swept capsule over interval I. In the box-based realization used below this is written as $B_\ell(I) = B^\circ_\ell(I) \oplus \mathcal{R}(r_\ell)$, where $B^\circ_\ell(I)$ bounds the centerline sweep and the padding operator $\mathcal{R}$ becomes B_∞ for AABB envelopes.

The following representations share the same endpoint evidence but preserve different amounts of downstream geometric detail.

a) AABB envelope.: AABB envelopes are the simplest option. The standard LinkIAABB construction takes the two endpoint envelopes, wraps them in one box, and pads that box by the link radius:

$$B_\ell(I) = \mathrm{bbox}\big(E_{\ell-1}(I) \cup E_\ell(I)\big) \oplus B_\infty(r_\ell).$$

This representation can be tightened by subdividing the reference center segment into S pieces, bounding each piece separately, and retaining the union of the padded sub-envelopes. Larger S preserves more directional variation along the link while keeping reconstruction efficient once endpoint envelopes are available. The main caveat is that the subdivision must remain explicit: if all sub-envelopes are collapsed back into one global AABB, much of the gain disappears.

b) SupportHull envelope.: When the endpoint envelopes are convex, their convex hull preserves directional structure that a single AABB discards. SupportHull is the compact runtime realization of that idea: instead of storing an explicit dense hull, it keeps the two endpoint AABBs and the link radius as a support-function record. For a query direction d, the support point is the better of the proximal and distal AABB support points, and collision refinement against an obstacle AABB uses a fixed-size Gilbert–Johnson–Keerthi (GJK) narrow phase with the obstacle inflated by the link radius and safety margin. This retains much of the convex- hull directional fidelity while keeping the record compact.

Thus the envelope layer is modular with respect to the upstream kinematics solver: endpoint records determine the link envelope, and the chosen representation sets the planning-time cost/tightness trade-off.

IV. Lifelong Envelope Cache Tree

The planner can materialize envelopes online; LECT is an optional replay layer for repeated builds or expensive envelope representations. It stores scene-independent endpoint/link-envelope evidence keyed by robot kinematics and joint intervals. Obstacle tests, box labels, graph connectivity, and query outcomes remain scene-local.

LECT is therefore separate from the scene-level RBF cache. The forest and corridor graph answer multiple queries in one fixed environment; LECT sits below that cache and reuses compatible kinematic evidence during builds, rebuilds, or local refills.

A. Cache Key and Split Policy

LECT is a dynamically grown binary KD tree over the joint-limit box $\mathcal{Q}$. Each node represents one joint interval and stores interval bounds, split metadata, cached endpoint evidence from which link envelopes can be reconstructed, and residency state for the runtime. The key property is that the stored evidence is *kinematics-keyed* rather than *scene-keyed*: later scenes may replay a materialized envelope record, but they must rerun their own scene-local obstacle tests. Nodes use a heap-style breadth-first indexing convention (0 at the root and $2p+1, 2p+2$ for children), which gives a finite indexed tree with configurable maximum depth $D_{\max}$.

a) Evidence/label separation.: An envelope record is replay-compatible only when the robot model, endpoint source, envelope representation, split descriptor, link radii, and active-link set match the current build. Replaying such a record imports endpoint and link-envelope evidence, not a free-space label. Every scene must still test the reconstructed link envelopes against its own obstacle set before a box can be accepted into the forest. This separation is what allows LECT to accelerate repeated geometry materialization without becoming a stale collision database.

The split policy is configurable and canonical for a given cache descriptor. The implemented schedules include round-robin, widest-root, and AAFK-volume-min. A build fixes one descriptor so that replay compatibility is well-defined; alternative descriptors are treated as separate cache settings. For a parent interval I, let I_d^- and I_d^+ be the two children obtained by splitting joint dimension d at its midpoint, and let $\mathcal{V}(I) := \sum_\ell \mathrm{vol}(B_\ell(I))$ be the cumulative downstream envelope volume. The AAFK-volume-min schedule chooses depth-wise split dimensions that minimize a worst-child envelope burden,

$$d^* = \arg\min_d \max\left\{ \sum_\ell \mathrm{vol}(B_\ell(I_d^-)), \sum_\ell \mathrm{vol}(B_\ell(I_d^+)) \right\}.$$

The resulting descriptor is depth-synchronous: all nodes at the same depth use the same scheduled dimension when that dimension is still wide enough, otherwise the runtime falls back to a valid widest dimension. Candidate dimensions may also be filtered by FK reachability, since splitting a joint beyond the last active frame that contributes to any link endpoint cannot tighten the endpoint envelopes.

B. Runtime Use and Validation Boundary

LECT is queried by FINDFREEBOX, not by the graph search itself. Given a joint interval, the cache either replays endpoint evidence or materializes it online, then derives the requested link representation. Directional records are used as refinement evidence rather than as standalone free-space certificates: the runtime first evaluates padded link AABBs, then uses SupportHull/GJK only for survivors of the AABB test. When a node is split, child envelopes may refine the parent witness, but any compact parent record remains representation-dependent and is rechecked against the current obstacle set before a box can enter the forest.

C. Storage and Warm-Build Semantics

Persistence is selective. Endpoint evidence is the common reusable record, but simple IFK+AABB evidence can be lower-cost to recompute online than to reload. SupportHull records are stored only when their tighter filters are expected to amortize storage and lookup cost. Reuse accounting should therefore attribute cache hits only to compatible kinematic evidence replay and not rely on additional symmetry-transfer assumptions.

The prewarm phase follows the same separation. It materializes FK/link-envelope evidence only on the target leaf layer and reconstructs upper layers by child-hull propagation during checkpoint. This avoids charging direct FK at multiple depths for the same cache and keeps parent witnesses tied to the leaf records that generated them. Exact leaf evidence and propagated child-hull evidence are both treated as envelope records by the replay path; provenance does not create a separate reuse or validation mode.

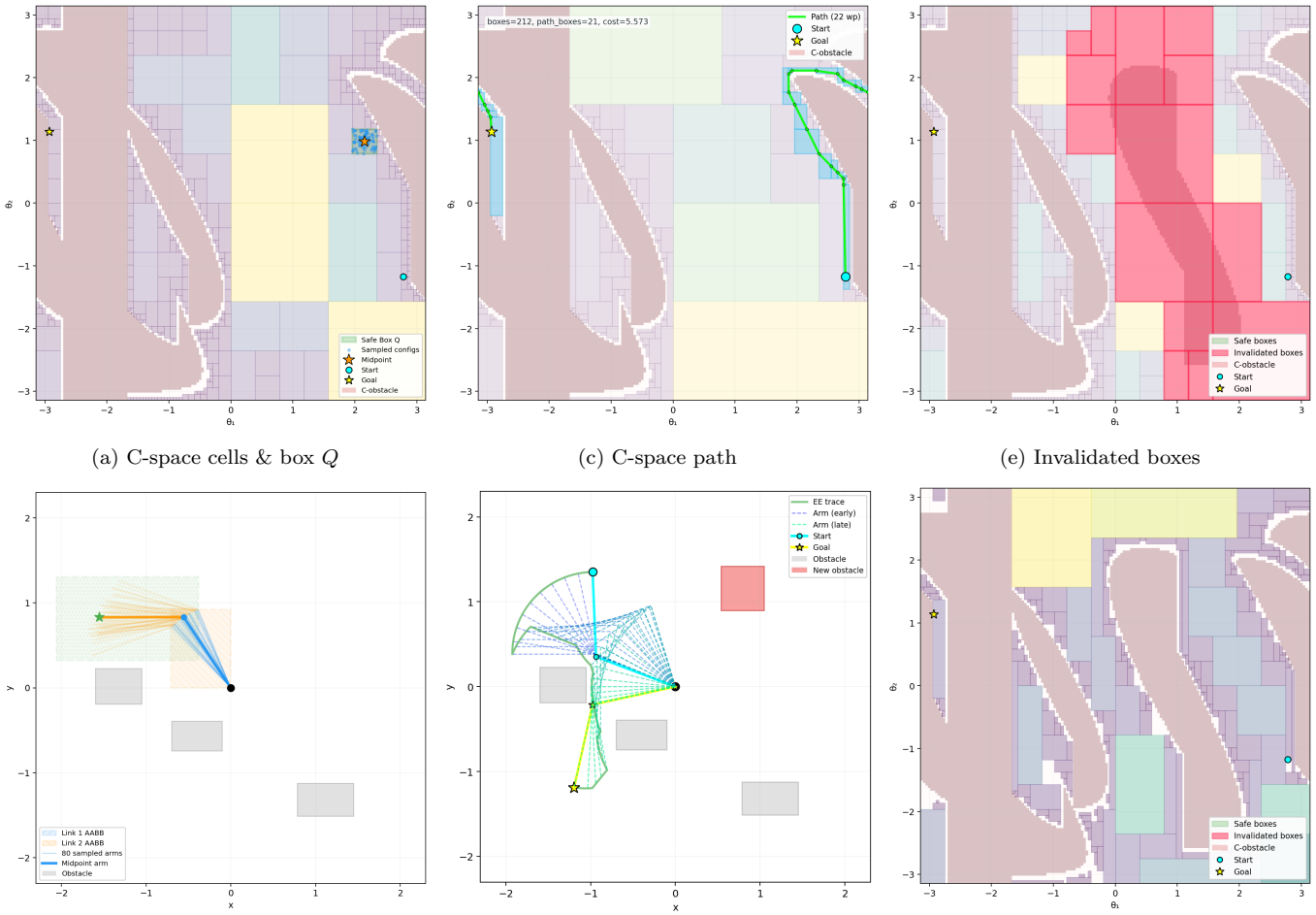
In the warm-build path, a cache hit reconstructs the requested link-envelope representation from stored endpoint records and revalidates it against the current obstacles. Warm builds still regrow scene-specific boxes; persisted box labels are not imported as certificates. Obstacle checking, adjacency, scene-level query reuse, and reactions to changing $\mathcal{O}$ remain in the forest routines. Fixed-replay studies can disable writeback, while cache-growth studies use a separate writeback policy.

V. RapidBoxForest Construction

This section describes the RBF planning layer. The algorithm grows a scene-specific forest of C-space boxes and exports it as a typed corridor graph for repeated queries. This forest-and-graph pair is the scene-level cache: while the obstacle set is unchanged, new queries use membership lookup and graph search instead of regrowing the forest. Samples choose frontiers, the envelope oracle validates boxes, and accepted boxes extend the connected region graph. The link interval envelopes of section III and LECT of section IV provide the kinematic evidence used by this scene-level decomposition.

a) Box, cell, and domain terminology.: In the planner, a *box* is an axis-aligned joint interval $B = \prod_d [l^{(d)}, u^{(d)}] \subseteq \mathcal{Q}$. The term *cell* is used when viewing the same object as an adaptive-decomposition element. A *collision domain* is a cell that has not been admitted to the forest under the scene-validation policy but is retained as a bounded refinement or update region. Thus boxes, cells, and domains share the same interval representation; the terms distinguish their role in the construction.

b) Validation terminology.: Throughout this section, a *conservatively validated box* is a joint box whose conservative link envelopes are disjoint from the current



(a) C-space cells & box Q (c) C-space path (e) Invalidated boxes

(b) Workspace AABBs of Q (d) Workspace path (f) After regrowth

Fig. 2. Illustrative RBF build, query, and obstacle-update cycle on a planar 2-link manipulator. (a, b) Adaptive C-space cells and workspace AABB envelopes of the highlighted box Q. (c, d) Box-corridor path in C-space and its workspace trajectory. (e, f) After obstacle insertion, intersecting boxes are invalidated (red), and localized refill plus graph repair updates the explicit forest without requiring a full rebuild.

obstacle set. These boxes support the formal corridor statement below. A *performance-mode box* is accepted by a tighter nonconservative filter and is therefore treated as a planning candidate until the recovered path passes query validation. We use *validated box* for the box admitted by the selected scene-validation policy. This policy is the box-admission rule used during forest construction: it may be the conservative envelope-disjointness test, a performance-mode filter, or the same test augmented with task-specific scene constraints. It is separate from the query-validation path, which checks recovered paths that do not inherit the conservative box-corridor certificate. We explicitly state when the stronger conservative certificate is required. Paths that leave the conservative box union through repair, smoothing, or external composition are charged where used and passed through the same query validation.

The build process has four stages. First, a seed-guided local oracle adaptively refines and classifies a C-space cell around a selected configuration. Second, the leaf-sweep mode constructs a shallow, reusable partition of the current scene and identifies conservative refinement domains. Third, repeated frontier expansion or restricted refinement grows validated cells into a connected forest, giving the method a region-valued, frontier-biased construction. Fourth, consolidation coarsens redundant boxes before the structure is exported as a query graph. The consolidation pass is part of the reusable build phase.

A. Construction Objective

Let $\mathcal{Q}$ denote the joint-limit box and let $\mathcal{O}$ denote the current obstacle set. The construction objective is to build a finite family

$$\mathcal{F} = \{B_1, \ldots, B_N\}, \qquad B_i \subseteq \mathcal{Q},$$

such that every B_i is admitted by the selected scene-validation policy, the union $\bigcup_i B_i$ covers as much query-relevant free space as possible under the available box, time, and depth budgets, and the induced adjacency graph supports efficient downstream search. In conservative mode these admitted boxes are formal free-space certificates; in performance-oriented modes they are planning regions whose recovered paths require query validation. This object is scene-specific: changing $\mathcal{O}$ changes which intervals are accepted into the forest, even though the underlying envelope evidence in LECT remains reusable.

The central structural rule is that the forest is not merely a free-space cover, but a *corridor-compatible* free-space cover. Boxes are inserted under a connected-box constraint.

Viewed as adaptive cell decomposition, this objective is intentionally workload-biased rather than globally exhaustive. The planner does not try to classify every cell in $\mathcal{Q}$. It refines and accepts cells near query-relevant frontiers, task seeds, and connector candidates, then keeps the resulting adjacency graph as the reusable planning object.

a) Connected-box rule.: For boxes

$$B_i = \prod_{d=1}^{n} [l_i^{(d)}, u_i^{(d)}], \qquad B_j = \prod_{d=1}^{n} [l_j^{(d)}, u_j^{(d)}],$$

RBF treats them as connected when, in every coordinate, the two intervals either overlap or are separated by at most a small tolerance ε_c:

$$\max\{l_i^{(d)}, l_j^{(d)}\} \le \min\{u_i^{(d)}, u_j^{(d)}\} + \varepsilon_c, \qquad d = 1, \dots, n.$$

When $\varepsilon_c = 0$, this reduces to standard interval intersection or boundary touching in every dimension. A small positive tolerance absorbs the boundary offset used by grower seeds and floating-point roundoff.

This rule is deliberately weaker than requiring a full shared face. It treats overlap, touching, and tolerance-scale gaps uniformly as graph-connectivity evidence, while validity remains attached to the accepted box union itself. Only overlapping or touching box chains directly imply a contained corridor; tolerance-gap edges are query candidates that must be bridged by contained waypoints or by explicit segment validation. The output of the construction stage is therefore a forest of validated boxes whose graph connectivity is aligned with the corridor object queried later, without overstating a stricter face-sharing invariant than the grower actually maintains.

B. Find Free Box

FINDFREEBOX descends LECT from the root toward the leaf containing seed $q \in \mathcal{Q}$, materializing endpoint and link envelopes lazily along the path, honoring reserved-node and skip-to-depth controls, and returning the first validated ancestor on the seed path. This is the largest canonical node on that path accepted by the scene-validation policy before further descent is needed. Tree topology and endpoint records are reused from the scene-independent cache, while scene-specific validation is performed on each grower visit. Algorithm 1 gives the pseudocode.

The termination behavior follows directly from the depth cap. Under $D_{\max}$, each loop iteration either returns immediately or descends one more level in the tree, so the procedure can make at most $D_{\max} + 1$ decisions. In strict conservative mode, any returned interval has passed envelope disjointness against $\mathcal{O}$; in performance-oriented modes, the returned interval is a validated planning region that remains subject to the common query-validation path.

Algorithm 1 FindFreeBox: local box materialization around a seed.

Input: Seed configuration q, LECT T, obstacle set $\mathcal{O}$, depth cap $D_{\max}$, skip depth D_{skip}
Output: Validated interval I_v or failure

$v \leftarrow \text{root}(T)$; $d \leftarrow 0$; initialize current interval and FK state
while $d \le D_{\max}$ **do**
  Cache materialization
  materialize the endpoint and link envelopes of v if needed
  Reservation and scene validation
  if v is reserved and cannot be split further **then**
    return failure
  end if
  if $d \ge D_{\text{skip}}$ and v passes the scene-validation policy **then**
    return interval I_v
  end if
  Refinement
  if $d = D_{\max}$ **then**
    return failure
  end if
  if v has no children **then**
    split v along the depth-synchronous split dimension of depth d
  end if
  $k \leftarrow$ split dimension of v
  $v \leftarrow$ the child of v whose interval contains q; update the current interval
  update the FK state from dimension k onward; $d \leftarrow d + 1$
end while
return failure

C. Leaf Sweep and Restricted Refinement

The fast construction path uses FINDFREEBOX inside a two-stage leaf-sweep routine. Rather than relying only on stochastic frontier proposals to discover query-relevant coarse regions, RBF first enumerates a fixed-depth set of LECT leaves and classifies them with the current obstacle set. Let $\mathcal{L}_{d_0:d_1}$ denote the virtual leaf layer obtained by expanding the tree from start depth d_0 to maximum sweep depth d_1. Each leaf $C \in \mathcal{L}_{d_0:d_1}$ is tested by the same link-envelope scene oracle used by FINDFREEBOX. A leaf that is certified free is inserted into the initial forest. A leaf that cannot be certified free is retained as a conservative *collision domain*. Such a domain denotes an inconclusive bounded region rather than a proof of occupancy; deeper refinement may still expose valid boxes inside it.

During this sweep, RBF also records a blocker set

$$\beta(C) \subseteq \{1, \dots, |\mathcal{O}|\}$$

for each non-free collision domain. This set is conservative update metadata: it contains obstacle groups that were observed as blockers or were not proven free by the grouped shallow sweep. The blocker set is an over-approximate update explanation; certification of boxes entering the forest remains governed by the standard envelope validation rule.

After the shallow sweep, restricted refinement is performed only inside selected collision domains. Domains are ranked by their adjacency to already certified boxes, volume, and proximity to query-priority configurations. Candidate seeds are placed at interfaces between a collision domain and neighboring free boxes, with additional center and boundary seeds to avoid depending on a single contact point. For a domain C, every accepted refined box B must satisfy

$$B \subseteq C$$

Algorithm 2 LeafSweepRefine: shallow coverage with restricted deep refinement.

Input: LECT T, obstacle set $\mathcal{O}$, sweep depths d_0, d_1, refinement budget N_r
Output: Validated forest $\mathcal{F}$ and collision-domain cache $\mathcal{C}$

$\mathcal{F} \leftarrow \emptyset$; $\mathcal{C} \leftarrow \emptyset$
for all virtual leaves $C \in \mathcal{L}_{d_0:d_1}$ **do**
  if C is certified free by the link-envelope scene oracle **then**
    insert C into $\mathcal{F}$
  else
    record C and its blocker set $\beta(C)$ in $\mathcal{C}$
  end if
end for
build adjacency among boxes in $\mathcal{F}$
for all collision domains $C \in \mathcal{C}$ in priority order **do**
  generate interface, center, and boundary seeds inside C
  for all seed q while budget remains **do**
    $B \leftarrow \text{FindFreeBoxInDomain}(q, T, \mathcal{O}, C)$
    if $B \neq \emptyset$, $B \subseteq C$, and B is adjacent to $\mathcal{F}$ or to a prior box from C **then**
      insert B into $\mathcal{F}$ and update local adjacency
    end if
  end for
end for
return $\mathcal{F}, \mathcal{C}$

and must be adjacent either to an existing forest box or to a box already accepted from the same domain. Thus the refinement stage can fill narrow passages revealed inside a shallow collision domain without spending its budget on unrelated regions of $\mathcal{Q}$. The connector stage may subsequently add collision-checked segment witnesses between remaining islands, but these witnesses are recorded separately from box-overlap edges.

D. Forest Growth

Starting from an initial seed set, the forest is grown by repeatedly invoking the find-free-box procedure on carefully chosen candidate configurations. Each seed first produces a root box through the find-free-box procedure, and these root boxes initialize one or more connected components of the forest. The main grower is an evidence-guided, frontier-biased expansion process rather than a generic random cover. Each iteration chooses a frontier component, proposes a nearby interval worth testing, and accepts the result only when it extends the forest by an admissible connected corridor segment. This keeps the build process focused on boxes that can improve query connectivity rather than on uniformly covering all of $\mathcal{Q}$.

Each iteration makes two explicit heuristic decisions: it samples a target configuration q_t, then chooses a boundary seed on an existing box face that is closest to that target. The target sampler is a configurable categorical mixture over intertree roots, unexplored-volume targets, query roots, fixed anchors, and uniform roots. Build connectivity is handled primarily by overlap-compatible box insertion; segment bridges remain a secondary mechanism recorded separately from box-overlap edges. The mixture weights, depth caps, cache depth, thread count, and anchor set are planner settings rather than envelope-theory parameters.

Given a target q_t and an existing box $B = \prod_j [\ell_j, u_j]$, the grower enumerates feasible faces $f = (B, d, s)$, where d is a joint dimension and $s \in \{-1, +1\}$ is the lower or upper side. Let τ be the adjacency tolerance and $\epsilon_b = \max(\epsilon_{\text{guard}}, \tau/4)$. A face is feasible only if stepping outside it by ϵ_b remains inside the global root interval. The seed associated with the face is

$$q_f[j] = \begin{cases} u_d + \epsilon_b, & j = d,\ s = +1, \\ \ell_d - \epsilon_b, & j = d,\ s = -1, \\ \text{clip}(q_t[j], \ell_j + \epsilon_b, u_j - \epsilon_b), & j \neq d, \end{cases} \tag{1}$$

with clipping to $[\ell_j, u_j]$ when a box is thinner than $2\epsilon_b$. Faces are ranked by the squared target distance

$$\sigma(f; q_t) = \|q_t - q_f\|_2^2, \tag{2}$$

and the grower retains a bounded candidate set of size K_{face} across the scanned frontier boxes. It then tries them in increasing score order, skipping a candidate if the corresponding LECT leaf is temporarily deferred or if a component-connection cache has already recorded the same boundary seed. The first remaining candidate becomes the seed for FINDFREEBOX.

The candidate box is accepted only if FINDFREEBOX validates it and the new box is connected to its parent under the same adjacency predicate used by the query graph: all dimensions overlap up to tolerance τ, and at least one dimension touches within tolerance or all dimensions overlap. Failures are also handled explicitly. When the LECT leaf containing a failed seed reaches the configured hard-frontier failure threshold K_{fail}, the leaf is skipped for a deferral horizon H_{cool} of subsequently accepted boxes, unless a later depth stage raises the active search depth and enables a retry. This deferral rule and the frontier-seed cache do not alter the validation boundary; they only reduce repeated calls on already failed or already covered frontier regions. The resulting method is summarized in algorithm 3.

The same construction also supports coordinated multi-goal growth. Multiple seed configurations can initialize roots in one authoritative forest, allowing the scene-level RBF cache to serve a batch of related queries while sharing LECT evidence across build workers and cache-warm reruns. Staged wavefront growth and task-batched growth are scheduling choices; they do not change the mathematical object produced by the planner, namely the query-ready box-region graph.

E. Forest Consolidation

Raw growth tends to produce a forest that is finer than necessary. Consolidation reduces this representation without changing the validation rule or the typed-edge semantics used by corridor search. The planner uses two conservative operations. First, connected pairs whose union is again a box may be merged after validation. Second, more general connected candidate pairs are ranked by a volume-expansion score and accepted only if the candidate hull passes the same scene oracle. A final containment-pruning pass removes boxes that are fully covered by accepted validated regions when their graph incidence can

Algorithm 3 GrowForest: budgeted expansion of a connected box-region forest.

Input: LECT T, obstacle set $\mathcal{O}$, seed set $\mathcal{S}$
Budgets/settings: $N_{\max}, t_{\max}, K_{\text{face}}, K_{\text{fail}}, H_{\text{cool}}$
Output: Validated box-region forest $\mathcal{F}$
$\mathcal{F} \leftarrow \emptyset$; initialize connected components from $\mathcal{S}$
for all $q \in \mathcal{S}$ **do**
  $B \leftarrow \text{FindFreeBox}(q, T, \mathcal{O})$
  if $B \neq \emptyset$ **then**
    insert B into $\mathcal{F}$ as a root box
  end if
end for
while $|\mathcal{F}| < N_{\max}$ and time budget not exhausted **do**
  draw target q_t from the configured connector/query/unexplored/uniform mixture
  enumerate feasible faces $f = (B, d, s)$ of frontier boxes and compute q_f and $\sigma(f; q_t)$
  rank the K_{face} lowest-score face seeds
  choose the first ranked seed that is not deferred and not in the frontier-seed cache
  set $(q_{\text{seed}}, B_{\text{near}}) \leftarrow (q_f, B)$ for that face
  $B_{\text{new}} \leftarrow \text{FindFreeBox}(q_{\text{seed}}, T, \mathcal{O})$
  if $B_{\text{new}} \neq \emptyset$ and $\text{BoxesConnected}(B_{\text{new}}, B_{\text{near}}, \tau)$ **then**
    insert B_{new} into $\mathcal{F}$; $\text{UpdateComponents}(\mathcal{F})$
  else
    record the failed LECT leaf; defer it after K_{fail} failures for horizon H_{cool}
  end if
end while
return $\mathcal{F}$

Algorithm 4 ConsolidateForest: graph-preserving merging and pruning.

Input: Validated forest $\mathcal{F}$, LECT T, obstacle set $\mathcal{O}$
Output: Consolidated validated forest $\mathcal{F}'$
$\mathcal{F}' \leftarrow \mathcal{F}$
Stage 1: validated connected/contact merging
merge exact connected pairs whose union is itself a single box
for all connected candidate pairs (B_i, B_j) in $\mathcal{F}'$ **do**
  build a candidate parent region H and score its volume expansion
  if the chosen envelope representation of H passes the scene-validation policy **then**
    replace B_i, B_j by H
  end if
end for
Stage 2: containment pruning
remove only contained boxes whose typed adjacency and segment witnesses transfer to selected covering regions
return $\mathcal{F}'$

be transferred. The consolidation logic is summarized in Algorithm 4.

Validity follows directly: every accepted merge has itself passed the scene oracle. Containment pruning removes only boxes fully covered by a selected region that inherits their typed adjacency and segment witnesses, preserving the connectivity semantics used by query search.

F. Corridor Query

After growth and consolidation, the forest is exported as a typed graph $G = (V, E_\cap \cup E_\varepsilon \cup E_s)$ with one vertex per validated box. Edges in $E_\cap$ connect boxes that overlap or touch in all dimensions; these edges directly preserve a contained box corridor. Edges in E_ε come only from tolerance-scale connected-box gaps and are treated as query candidates rather than as corridor certificates. Edges in E_s are explicit collision-checked segment witnesses. Because typed connectivity is maintained during growth and consolidation, online querying reduces to membership lookup and typed graph search rather than post-hoc graph reconstruction.

If disconnected adjacency islands remain after growth, an optional connector stage attempts targeted bidirectional sampling bridges between nearest frontier boxes of different components. A successful bridge is stored as a segment edge rather than as a claim that the two boxes overlap. This distinction is important for both the RBF query layer and the GCS negative controls: a segment edge is a collision-checked path witness between two boxes, whereas a GCS edge over convex regions requires feasible overlap of the regions themselves.

This pair $(\mathcal{F}, G)$ is the planning object produced by the build stage. It is also the scene-level RBF cache for subsequent queries in the same environment. Given start and goal configurations q_s, q_g, the planner associates them with start-side and goal-side boxes in $\mathcal{F}$ and then searches G for a connecting box sequence. The online query problem is reduced to box-corridor retrieval inside an already constructed box-region representation.

A shortest path on G yields a candidate box sequence between the two query endpoints. When the recovered geometric path stays inside the union of conservatively validated boxes, it is certified collision-free by theorem 1. Paths that use E_ε or E_s edges, performance-mode boxes, or post-processing steps that leave the conservative box union are treated through the configured query-validation path, and the query interface records these cases through the `QueryResult` counters. The box-corridor statement below applies to paths contained in that union.

For an $E_\cap$-only graph path, this containment condition has a direct constructive interpretation. Each vertex box is convex, and each $E_\cap$ edge means that consecutive boxes overlap or touch. Therefore a continuous piecewise-linear curve can be chosen by selecting one point in each nonempty pairwise intersection and connecting these points inside the corresponding boxes. If q_s and q_g lie in the first and last boxes, respectively, the same convexity argument connects them to the first and last intersection points. The resulting curve is contained in the union of the boxes on the graph path. This is a containment certificate, not a positive-clearance certificate: touching boxes may meet at a lower-dimensional set, and clearance depends on the underlying envelope-obstacle separation or on the configured query-validation test. This construction does not apply to E_ε or E_s edges, whose semantics are candidate gap closure and explicit segment witnessing, respectively.

Theorem 1 (Conditional conservative box-corridor path)**.** *Let $\mathcal{F}$ be a forest whose boxes have each passed conservative link-envelope validation against obstacle set $\mathcal{O}$. If a continuous path $\gamma : [0,1] \to \mathcal{Q}$ satisfies $\gamma([0,1]) \subseteq \bigcup_{B \in \mathcal{F}} B$, then every configuration on γ is collision-free with respect to the collision model in assumption 1.*

Proof. For any $s \in [0,1]$, the containment assumption gives a conservatively validated box $B \in \mathcal{F}$ with $\gamma(s) \in B$.

By construction, validation of B means every conservative link envelope for B is disjoint from $\mathcal{O}$. Because each such envelope contains the link geometry for every configuration inside B (corollary 1), the robot geometry at $\gamma(s)$ is also disjoint from $\mathcal{O}$. Since s was arbitrary, the whole path is collision-free. □

a) Reporting protocol.: Theorem 1 is a conditional statement: it covers path portions contained in a conservatively validated box union. Post-processing inherits this certificate only when it preserves that containment condition. The result is therefore a *soundness* statement for the returned box-contained corridor, not an unconditional claim that a finite-budget forest covers every feasible passage in $\mathcal{Q}$. Coverage is controlled by the selected anchor set, subdivision depth, refinement budget, and update policy. In the conservative mode, any path retained inside the accepted conservative box union inherits the certificate above; paths that require uncovered regions must wait for additional refinement, a larger forest budget, or an explicit query-validation path.

G. Dynamic Obstacle Updates

The explicit forest representation also supports local typed-graph maintenance under obstacle edits. The update procedure assumes fixed robot kinematics and a fixed LECT topology/evidence cache; only the obstacle set and scene-dependent forest labels and typed edges change. The key distinction is asymmetric: removing obstacles can expose previously blocked cached domains, whereas inserting obstacles may invalidate boxes and segment witnesses that were previously accepted.

The update invariant is local but explicit: every box retained or inserted after an edit must pass the edited-scene validation policy, every $E_\cap$ edge must still correspond to overlap or contact of its endpoint boxes, every E_ε edge remains only a tolerance-gap query candidate, and every E_s edge must keep a valid segment witness in the edited scene. The update procedure is therefore a maintenance rule for the represented forest and its typed graph, not a claim of complete rediscovery of all newly available free-space regions after an edit.

For obstacle deletion, the blocker sets $\beta(C)$ recorded during the leaf-sweep stage provide a conservative candidate-selection rule for the information that was actually cached. When obstacle indices R are removed, each cached collision domain C updates its blocker set to $\beta(C) \setminus R$, with the remaining obstacle indices remapped to the edited scene. If this set becomes empty, the domain becomes eligible for promotion. Promotion either reuses sufficient cached disjointness evidence for the edited obstacle set or reruns the standard scene check on the reconstructed envelope record. It is also filtered by containment and adjacency checks: contained candidates are discarded, disconnected candidates remain cached, and accepted candidates are inserted into the forest and connected incrementally. Thus deletion updates use the recorded blocker cache to focus validation effort and are limited to domains represented in that cache. Discovering additional newly free regions is left to a subsequent build or refinement stage.

Algorithm 5 DynamicUpdate: obstacle deletion promotion and insertion refill.

Input: Forest $\mathcal{F}$, typed graph G, collision-domain cache $\mathcal{C}$, edit (Δ^-, Δ^+)
Output: Updated forest, typed graph, and collision-domain cache

if $\Delta^- \neq \emptyset$ **then**
  for all $C \in \mathcal{C}$ **do**
    remove deleted obstacle indices from $\beta(C)$ and remap surviving indices
    if $\beta(C) = \emptyset$ **then**
      promote C to $\mathcal{F}$ only if containment, adjacency, and edited-scene validation pass
    end if
  end for
  incrementally connect promoted boxes to overlapping or touching forest boxes
end if
if $\Delta^+ \neq \emptyset$ **then**
  $\mathcal{I} \leftarrow \{B \in \mathcal{F} : B$ conflicts with some obstacle in $\Delta^+\}$
  remove $\mathcal{I}$ and typed graph/segment witnesses that fail edited-scene validation
  for all $B \in \mathcal{I}$ **do**
    set the local sweep depth, optionally capped relative to depth(B)
    sweep leaves inside B to that local depth
    insert swept leaves that pass edited-scene validation and are adjacent to the surviving forest
    cache still-blocked leaves with blockers in Δ^+
  end for
  incrementally connect newly inserted boxes and revalidate surviving segment edges
end if
if the requested query is still disconnected **then**
  optionally run endpoint recovery and collision-checked segment recovery
end if
return updated $(\mathcal{F}, G, \mathcal{C})$

Obstacle insertion uses the complementary path. Each added obstacle is first tested against the current live boxes, optionally restricted by the spatial dirty region. Boxes that collide with the added-obstacle checker are removed from the forest and released from the oracle reservation map; raw boxes are checked and removed by the same edited-scene logic. Segment edges whose endpoints disappear or whose waypoint witness fails the edited-scene validation are removed as well. Each invalidated box then becomes a bounded refill domain. The insertion routine performs a local leaf sweep inside this domain up to the configured insertion depth, optionally capped relative to the invalidated box depth. Leaves that pass the scene-validation policy are reinserted; leaves that remain blocked or hit the depth cap are cached with a blocker set containing the new obstacle index. The insertion update is therefore a local invalidation-and-refill operation over the typed graph: invalidated boxes are removed, local leaf-sweep refill proposes replacement boxes, affected segment witnesses are revalidated, and typed connectivity is repaired incrementally. If the edited graph still cannot answer the query, the query pipeline may use its configured repair or segment-edge mechanisms.

If no contained conservative corridor is recovered, the box-corridor certificate does not apply; repaired, post-processed, or segment-recovered paths are accepted only

through query validation.

VI. Experiments

This section evaluates RBF as a reusable planning system under the current registered protocol. All paper-facing experiment scripts live in `experiments/`; the legacy `sbf_old` scripts are archival references only. Tables and figures in this section are generated into `paper/generated/` from JSON/CSV artifacts, with provenance recorded in `tro_table_generation_manifest.json`.

a) Common success and timing semantics.: Planner success means that the returned path passes the fixed-resolution query-validation check. Validation time is reported separately and is not charged to planning time. Path-length statistics are success-only means, while success rate is reported over all registered runs. Query-time shortcut boxes and final collision shortcutting are disabled for the main no-post RBF rows.

b) Registered RBF default.: Unless otherwise stated, RBF uses the Exp. 4 profile `exp04_leaf_sweep_rrt_b200_d28_fast_rrt1`: a virtual leaf sweep from depth 8 to 14, followed by restricted deep refinement inside shallow collision domains and a bounded RRT grower. The default deep budget is 200 boxes, FINDFREEBOX uses maximum depth 28, the RRT grower is kept as a low-budget connectivity repair stage with a one-box, 1 ms cap, and virtual leaf validation uses the registered parallel worker setting. Build-stage RRT segment witnesses are enabled and explicitly accounted. The connector is capped at one 50 ms pair attempt per gap. Connector segment validation uses the same 0.01 rad maximum joint-space sample spacing as the final strict audit, so a segment witness cannot be accepted under a looser build-time collision check. Shelf+IIWA baseline rows additionally use the `exp04_leaf_sweep_rrt_d23_b200_d28_fast_rrt1` warm-cache override, where the d23 external LECT evidence cache is read-only. Random-scene rows use the same algorithm defaults but robot-local roots and no Shelf anchors or Shelf d23 cache.

A. Shelf+IIWA Leaf-Sweep–RRT Trade-off

Figure 3 and table II show the Shelf+IIWA trade-off over seeds 0–7 and the five canonical query pairs. The tabulated rows use the common-rule design points selected from their budget curves, while the figure shows the complete budget sweep. A CritSample row is included as a sampling-based upper-reference ablation: it can improve coverage and path quality, but it does not provide box-level safety certificates and is therefore not the registered baseline.

B. Shelf Cross-Algorithm Context

The Shelf+IIWA cross-algorithm experiment uses the same five canonical queries and 0.01 rad final-audit setting for RBF, IRIS-NP+GCS, PRM, RRTConnect, and BIT*. RBF rows are regenerated from the current Exp. 4 registered leaf-sweep–RRT budget curve, so table III reports the same registered RBF default as table II. The non-RBF rows are imported from the audited old TRO artifact only after checking the scene, query set, and final-audit resolution. IRIS-NP+GCS is assigned the larger old-paper region budget rather than the reduced diagnostic budget. The comparison is therefore a latency/amortization context for the reusable box-region graph, not a claim that RBF dominates one-shot planners in path length.

C. Random Multi-Robot Scenes

The random-scene experiment uses a saved v5 catalog generated by the current scene generator and then reused by every current RBF run. The generator samples actual start and goal configurations inside the union of reflected canonical LECT-root sections, stores their canonical representatives, rejects direct-line trivial cases, and requires an independent collision-checked RRT-Connect probe before a scene is admitted to the catalog. Scenes are stratified by robot (IIWA, UR5, Panda), difficulty (easy, medium, hard), and eight saved scene seeds; the `balanced_independent` profile uses difficulty-specific generator seeds, so the three difficulty strata are not shared prefixes of a single query set. Random-scene RBF rows do not reuse Shelf+IIWA anchors or the Shelf d23 cache; UR5 and Panda use robot-local d20 LECTDB caches, while IIWA uses the registered d23 LECTDB root. The table reports one readable current RBF trade-off point per robot–difficulty stratum: among full-success budget points within 8% of the shortest audited median path, it selects the fastest point. Figure 5 shows the complete 36-row current RBF box-budget curve. PRM, RRTConnect, and BIT* are also rerun on the same saved catalog; all 216 current OMPL baseline runs pass the same final audit. IRIS-NP+GCS remains imported from the old balanced random-scene common-rule artifact because that external Drake/GCS pipeline is not yet self-contained in the current runner. The current data show that RBF generalizes its audited box-region construction across robots, but it does not dominate single-query planners on random scenes: BIT* and RRTConnect often produce shorter paths or lower one-shot latency. All 288 current RBF random-scene runs in the saved catalog pass the common 0.01 rad final audit, and the selected current RBF rows use only box-overlap graph paths (zero segment-witness fraction).

D. Mechanism Studies

The mechanism experiments isolate the geometric and cache components from the planner. The endpoint-envelope study compares endpoint sources used by the link-envelope layer; its MC row uses a fixed sample density per joint-box geometric-mean width, and its negative-gap column reports the worst shortfall against the CritSample/Analytical/MC sampling-union reference. Analytical is run with the full phase-3 pipeline even at narrow widths, so the table reflects the certified analytical

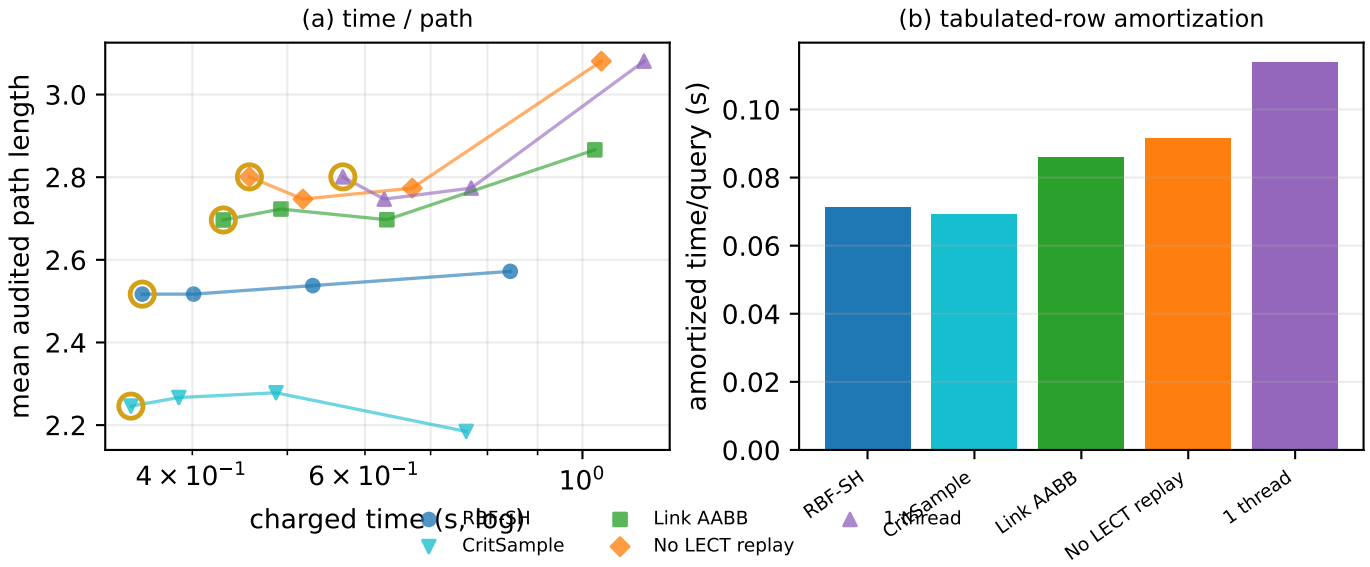


Fig. 3. Shelf+IIWA RBF ablation trade-off under the registered leaf-sweep–RRT profile. Panel (a) reports the final-audited time/path trade-off for the complete box-budget curves; panel (b) reports the amortized per-query cost for the tabulated design points. Gold rings mark the rows reported in table II.

TABLE II
Shelf+IIWA RBF ablation rows from Fig. 3, reported by query. Each ablation contributes the common-rule design point selected from its box-budget curve; the full curves remain the primary evidence. Query path entries are success-only means over fixed 0.01 joint-space strict-audit paths. The CritSample row is sampling-based and does not provide box-level safety certificates.

	RBF-SH b100 Build 0.203 s; SR 8/8			**CritSample b100** Build 0.202 s; SR 8/8			**Link AABB b100** Build 0.245 s; SR 8/8			**No LECT replay b100** Build 0.254 s; SR 8/8			**1 thread b100** Build 0.373 s; SR 8/8		
Query	Query (s)	Path	Seg.	Query (s)	Path	Seg.	Query (s)	Path	Seg.	Query (s)	Path	Seg.	Query (s)	Path	Seg.
AS→TS	0.074	2.43	0.00	0.053	2.26	0.00	0.094	2.63	0.00	0.105	2.84	0.00	0.101	2.84	0.00
TS→CS	0.004	1.10	0.00	0.001	0.75	0.00	0.004	1.10	0.00	0.004	1.10	0.00	0.004	1.10	0.00
CS→LB	0.004	2.27	0.06	0.017	2.47	0.00	0.034	2.82	0.10	0.036	2.82	0.10	0.035	2.82	0.10
LB→RB	0.042	3.09	0.04	0.034	2.68	0.00	0.007	2.79	0.24	0.008	3.19	0.21	0.007	3.19	0.21
RB→AS	0.028	3.69	0.00	0.039	3.07	0.00	0.045	4.15	0.09	0.051	4.05	0.09	0.050	4.05	0.09

TABLE IV
Saved-catalog random-scene best trade-off points. RBF rows are selected from the current v5 saved catalog; non-RBF columns use current saved-catalog rows when available and otherwise fall back to old balanced random-scene common-rule context. Full current RBF budget curves are shown in Fig. 5.

Scenario	Current RBF			IRIS-NP+GCS		PRM		RRTConnect		BIT*	
	Plan (s)	Path	Boxes	Time (s)	Path	Time (s)	Path	Time (s)	Path	Time (s)	Path
IIWA-Easy	1.087	2.71	800	76.400	5.46	2.411	3.28	0.001	2.56	1.006	2.36
IIWA-Medium	0.718	3.25	400	137.700	5.56	2.408	3.48	0.001	2.85	1.005	2.61
IIWA-Hard	0.500	3.32	200	187.500	5.46	5.239	3.34	0.001	2.77	1.006	2.89
UR5-Easy	0.449	9.09	400	18.140	7.62	1.416	5.17	0.002	4.92	1.006	4.69
UR5-Medium	0.273	10.50	100	29.160	7.60	0.671	7.26	0.007	16.58	1.006	5.02
UR5-Hard	0.285	10.67	100	40.670	7.62	0.920	6.21	0.003	9.52	1.005	4.66
PANDA-Easy	0.248	6.90	100	18.130	5.82	1.420	4.64	0.001	4.38	1.005	4.39
PANDA-Medium	0.317	7.70	100	30.120	5.87	1.419	4.82	0.001	4.44	1.005	4.90
PANDA-Hard	0.364	7.68	200	44.300	5.79	1.420	4.73	0.001	3.90	1.005	4.08

cost rather than a narrow-box shortcut. The link-envelope study fixes split count $S = 1$ and compares LinkIAABB, SupportHull, and staged cascades. The LECT study reports read-snapshot operation costs on a persisted robot cache, separating kinematics-keyed evidence replay from scene-dependent free-box labels.

E. Dynamic Updates

The dynamic-update experiment uses saved random scenes and deterministic obstacle insertion/deletion transitions. Incremental updates are compared against a fresh warm rebuild using the same registered leaf-sweep–RRT default. The reported metrics separate invalidated boxes, promoted domains, update time, warm rebuild time, validated-query success, and raw segment fraction. In the pilot run, all 32 edited-scene queries passed strict audit. Two insertion cases used the configured endpoint segment recovery after the incremental graph failed the first audited query; the median raw segment fraction remains zero for every transition and the maximum audited segment

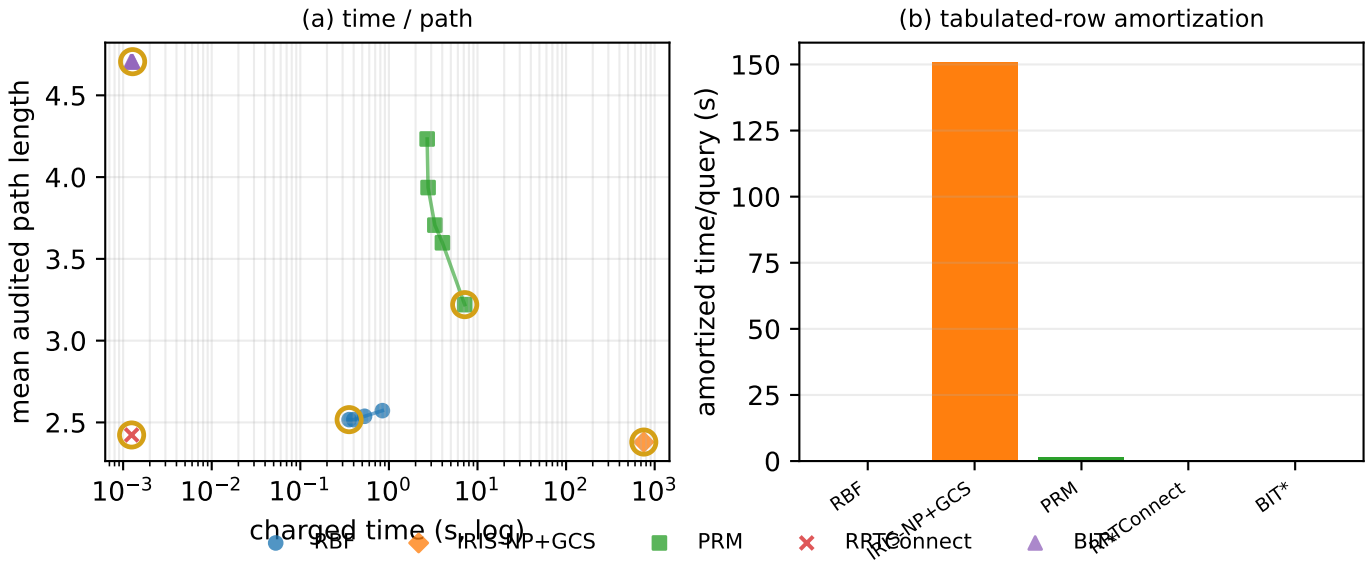


Fig. 4. Shelf+IIWA final-audited cross-algorithm trade-off. The RBF curve is regenerated from the current Exp. 4 registered profile; PRM, RRTConnect, and BIT* are current reruns with the old TRO baseline budgets, while IRIS-NP+GCS uses the audited old TRO artifact. Panel (a) reports complete trade-off curves and panel (b) reports the amortized per-query cost for the tabulated rows. Gold rings mark the rows reported in table III.

TABLE III
Common-rule tabulated Shelf+IIWA rows from Fig. 4, reported by query. Gold rings in the figure mark these rows only to expose detailed numeric values; the full curves remain the primary evidence. RBF uses the current leaf-sweep–RRT grower profile. PRM, RRTConnect, and BIT* are current reruns with old TRO baseline budgets; the IRIS-NP+GCS per-query entries are the audited old TRO artifact. OMPL planning, final simplify, and fixed-resolution final audit use the same 0.01 joint-space segment step.

	RBF b100 Build 0.356 s; SR 8/8		**IRIS-NP+GCS** Build 753.000 s; SR 1/1		**PRM** Build 6.764 s; SR 8/8		**RRTConnect** Build 0.006 s; SR 8/8		**BIT*** Build 0.007 s; SR 8/8	
Query	Query (s)	Path	Query (s)	Path	Query (s)	Path	Query (s)	Path	Query (s)	Path
AS→TS	0.074	2.43	0.558	2.38	0.409	3.22	0.001	2.30	0.001	4.59
TS→CS	0.004	1.10	0.463	1.99	0.410	0.69	0.001	1.18	0.001	0.69
CS→LB	0.004	2.27	0.533	2.63	0.411	2.02	0.001	2.74	0.001	4.82
LB→RB	0.042	3.09	0.825	3.26	0.345	4.32	0.001	2.55	0.001	6.03
RB→AS	0.028	3.69	0.765	1.84	0.407	4.80	0.001	4.59	0.001	5.33

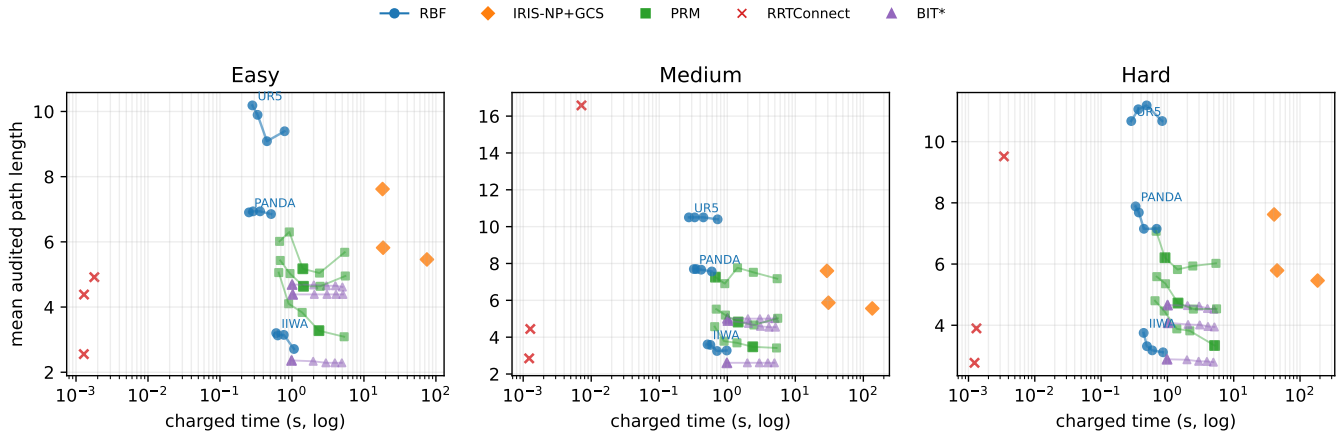


Fig. 5. Saved-catalog random-scene trade-off. Blue curves are current RBF budget sweeps on the saved v5 catalog. PRM, RRTConnect, and BIT* markers are rerun on the same saved catalog with the same 0.01 rad final audit. The IRIS-NP+GCS markers are imported from the old balanced random-scene common-rule artifact as protocol context.

fraction is 0.198.

VII. Conclusion

This paper presented RBF, a link-envelope-guided box-forest planner for multi-query manipulation. RBF lifts endpoint-to-link swept-capsule bounds from collision detection to C-space box validation, grows accepted boxes into a scene-level corridor graph, and separates that scene cache from LECT's lifelong kinematics-keyed envelope evidence. The design trades convex-region expressivity

TABLE V
Endpoint AABB source comparison at fixed joint-box widths. Cert. marks certificate-backed sources. MC uses width-density sampling, and Max neg. gap reports the worst per-axis shortfall against the CritSample/Analytical/MC sampling-union reference.

Width	Source	Cert.	V_{ep} (m^3)	Time (μs)	Max neg. gap
0.02	IFK_AA	Y	0.000045	5.5	-0.0000
0.02	HIFK_3	Y	0.000042	25.9	-0.0000
0.02	HIFK_5	Y	0.000042	70.4	-0.0000
0.02	CritSample	N	0.000041	8.7	-0.0000
0.02	Analytical	Y	0.000041	6430.0	-0.0000
0.02	MC	N	0.000026	2251.7	-0.0061
0.05	IFK_AA	Y	0.000797	6.1	-0.0000
0.05	HIFK_3	Y	0.000714	26.9	-0.0000
0.05	HIFK_5	Y	0.000690	71.7	-0.0000
0.05	CritSample	N	0.000647	9.0	-0.0003
0.05	Analytical	Y	0.000647	6500.4	-0.0000
0.05	MC	N	0.000454	5629.2	-0.0146
0.10	IFK_AA	Y	0.007709	6.5	-0.0000
0.10	HIFK_3	Y	0.006187	29.2	-0.0000
0.10	HIFK_5	Y	0.005748	73.8	-0.0000
0.10	CritSample	N	0.005085	10.0	-0.0012
0.10	Analytical	Y	0.005086	6602.3	-0.0000
0.10	MC	N	0.003739	11233.3	-0.0327
0.20	IFK_AA	Y	0.095828	6.6	-0.0000
0.20	HIFK_3	Y	0.063170	27.2	-0.0000
0.20	HIFK_5	Y	0.054478	73.1	-0.0000
0.20	CritSample	N	0.041680	11.6	-0.0045
0.20	Analytical	Y	0.041748	6800.1	-0.0000
0.20	MC	N	0.032592	22353.0	-0.0656
0.30	IFK_AA	Y	0.501814	6.8	-0.0000
0.30	HIFK_3	Y	0.268384	29.3	-0.0000
0.30	HIFK_5	Y	0.214676	75.9	-0.0000
0.30	CritSample	N	0.141227	12.7	-0.0108
0.30	Analytical	Y	0.142112	6927.1	-0.0000
0.30	MC	N	0.112648	33813.6	-0.0889
0.50	IFK_AA	Y	5.602815	7.4	-0.0000
0.50	HIFK_3	Y	2.028027	29.4	-0.0000
0.50	HIFK_5	Y	1.347150	72.3	-0.0000
0.50	CritSample	N	0.639900	16.3	-0.0310
0.50	Analytical	Y	0.650907	7096.0	-0.0045
0.50	MC	N	0.534496	56835.6	-0.1235

for low-cost validation, simple graph maintenance, and explicit reuse boundaries.

The guarantee is conditional: paths are certified only when they remain inside a union of conservatively validated boxes (theorem 1). Finite-budget forests need not cover all feasible passages, and repaired or post-processed paths require query validation. Future work will study broader scene distributions, stronger cache transfer, containment-preserving smoothing, and deeper comparisons with high-quality convex-region planners.

TABLE VI
Width-wise IFK_AA link-envelope comparison with one link split ($S = 1$). Entries are means over fixed-width boxes. Env. reports envelope materialization only; Coll. is the standalone obstacle test.

Width	Envelope	V (m^3)	Env. (μs)	Coll. (μs)
0.02	LinkIAABB	0.013628	0.1	0.3
0.02	SupportHull	0.000359	0.2	2.0
0.05	LinkIAABB	0.020148	0.1	0.4
0.05	SupportHull	0.002662	0.2	2.1
0.10	LinkIAABB	0.038415	0.2	0.3
0.10	SupportHull	0.014329	0.2	2.4
0.20	LinkIAABB	0.146513	0.1	0.4
0.20	SupportHull	0.105919	0.2	3.2
0.30	LinkIAABB	0.507549	0.2	0.4
0.30	SupportHull	0.448860	0.2	5.2
0.50	LinkIAABB	4.607121	0.2	0.3
0.50	SupportHull	4.537899	0.2	11.2

TABLE VII
LECT read-snapshot microbenchmark on a persisted cache (UR5 d20 SupportHull cache; 2,097,151 nodes, 2,097,151 evidence records).

Operation	Ops	Avg. (μs)	Throughput (Mops/s)
Open snapshot	1	86.22	0.01
Node box	20000	1.08	0.92
Exact lookup	20000	0.85	1.18
Endpoint lookup	20000	1.86	0.54
Endpoint lookup (hot)	20000	0.58	1.73
Range query	20000	22.60	0.04
Evidence read	20000	1.26	0.79
Evidence read (hot)	20000	0.13	7.56

TABLE VIII
Saved-catalog dynamic-update pilot results. Update time is compared with a fresh warm rebuild on the target scene; final audit time is excluded.

Transition	SR	Update (s)	Warm (s)	Speedup
easy->medium	8/8	0.045	0.549	14.346
hard->medium	8/8	0.006	0.549	85.692
medium->easy	8/8	0.004	0.634	138.987
medium->hard	8/8	0.059	0.444	10.738

References

[1] R. Deits and R. Tedrake, "Computing large convex regions of obstacle-free space through semidefinite programming," in *Algorithmic Foundations of Robotics XI*, 2015, pp. 109–124.

[2] T. Marcucci, M. Petersen, D. von Wrangel, and R. Tedrake, "Motion planning around obstacles with convex optimization," *Sci. Robot.*, vol. 8, no. 84, 2023.

[3] T. Marcucci, J. Umenberger, P. Parrilo, and R. Tedrake, "Shortest paths in graphs of convex sets," *SIAM J. Optim.*, vol. 34, no. 1, pp. 507–532, 2024.

[4] S. M. LaValle, *Planning Algorithms*. Cambridge University Press, 2006.

[5] ——, "Rapidly-exploring random trees: A new tool for path planning," Iowa State University, Tech. Rep. TR 98-11, 1998.

[6] L. E. Kavraki, P. Švestka, J.-C. Latombe, and M. H. Overmars, "Probabilistic roadmaps for path planning in high-dimensional configuration spaces," *IEEE Trans. Robot. Autom.*, vol. 12, no. 4, pp. 566–580, 1996.

[7] S. Karaman and E. Frazzoli, "Sampling-based algorithms for optimal motion planning," *Int. J. Robot. Res.*, vol. 30, no. 7, pp. 846–894, 2011.

[8] A. Amice, H. Dai, P. Werner, A. Zhang, and R. Tedrake, "Finding and optimizing certified, collision-free regions in configuration space for robot manipulators," in *Algorithmic Foundations of Robotics XV*. Springer, 2022, pp. 328–348.

[9] M. Petersen and R. Tedrake, "Growing convex collision-free regions in configuration space using nonlinear programming," 2023. [Online]. Available: https://arxiv.org/abs/2303.14737

[10] P. Werner, A. Amice, T. Marcucci, D. Rus, and R. Tedrake, "Approximating robot configuration spaces with few convex sets using clique covers of visibility graphs," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2024, pp. 10 359–10 365.

[11] D. Blackmore and M. C. Leu, "Analysis of swept volume via Lie groups and differential equations," *Int. J. Robot. Res.*, vol. 11, no. 6, pp. 516–537, 1992.

[12] K. Abdel-Malek, J. Yang, D. Blackmore, and K. Joy, "Swept

volumes: foundation, perspectives, and applications," *Int. J. Shape Model.*, vol. 12, no. 1, pp. 87–127, 2006.
[13] S. Gottschalk, M. C. Lin, and D. Manocha, "OBBTree: A hierarchical structure for rapid interference detection," in *ACM SIGGRAPH*, 1996.
[14] E. Larsen, S. Gottschalk, M. C. Lin, and D. Manocha, "Fast proximity queries with swept sphere volumes," University of North Carolina at Chapel Hill, Tech. Rep. TR99-018, 1999.
[15] E. G. Gilbert, D. W. Johnson, and S. S. Keerthi, "A fast procedure for computing the distance between complex objects in three-dimensional space," *IEEE J. Robot. Autom.*, vol. 4, no. 2, pp. 193–203, 1988.
[16] G. v. d. Bergen, "A fast and robust GJK implementation for collision detection of convex objects," *J. Graph. Tools*, vol. 4, no. 2, pp. 7–25, 1999.
[17] J. Pan, S. Chitta, and D. Manocha, "FCL: A general purpose library for collision and proximity queries," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2012, pp. 3859–3866.
[18] R. E. Moore, R. B. Kearfott, and M. J. Cloud, *Introduction to Interval Analysis*. SIAM, 2009.
[19] L. Jaulin, M. Kieffer, O. Didrit, and E. Walter, *Applied Interval Analysis*. Springer, 2001.
[20] J. M. Snyder, "Interval analysis for computer graphics," in *Proc. ACM SIGGRAPH*, 1992, pp. 121–130.
[21] J.-P. Merlet, "Solving the forward kinematics of a Gough-type parallel manipulator with interval analysis," *Int. J. Robot. Res.*, vol. 23, no. 3, pp. 221–235, 2004.
[22] ——, "Interval analysis for certified numerical solution of problems in robotics," *Int. J. Appl. Math. Comput. Sci.*, vol. 19, no. 3, pp. 399–412, 2009.
[23] S. Redon, Y. J. Kim, M. C. Lin, and D. Manocha, "Interactive and continuous collision detection for avatars in virtual environments," in *IEEE Virtual Reality Conference*, 2004, pp. 117–124.
[24] S. Redon, M. C. Lin, D. Manocha, and Y. J. Kim, "Fast continuous collision detection for articulated models," *J. Comput. Inf. Sci. Eng.*, vol. 5, no. 2, pp. 126–137, 2005.
[25] X. Zhang, S. Redon, M. Lee, and Y. J. Kim, "Continuous collision detection for articulated models using Taylor models and temporal culling," *ACM Trans. Graph.*, vol. 26, no. 3, p. 15, 2007.
[26] F. Schwarzer, M. Saha, and J.-C. Latombe, "Adaptive dynamic collision checking for single and multiple articulated robots in complex environments," *IEEE Trans. Robot.*, vol. 21, no. 3, pp. 338–353, 2005.
[27] J. A. Corrales, F. A. Candelas, and F. Torres, "Safe human–robot interaction based on dynamic sphere-swept line bounding volumes," *Robot. Comput.-Integr. Manuf.*, vol. 27, no. 1, pp. 177–185, 2011.
[28] R. Bohlin and L. E. Kavraki, "Path planning using lazy PRM," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2000, pp. 521–528.
[29] C. M. Dellin and S. S. Srinivasa, "A unifying formalism for shortest path problems with expensive edge evaluations via lazy best-first search over paths with edge selectors," in *International Conference on Automated Planning and Scheduling (ICAPS)*, 2016.
[30] E. Schmerling, L. Janson, and M. Pavone, "Optimal sampling-based motion planning under differential constraints: The drift-less case," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2015.
[31] D. Coleman, I. Sucan, S. Chitta, and N. Correll, "Reducing the barrier to entry of complex robotic software: A MoveIt! case study," *J. Softw. Eng. Robot.*, vol. 5, no. 1, pp. 3–16, 2014.
[32] T. Lozano-Pérez, "Spatial planning: A configuration space approach," *IEEE Trans. Comput.*, vol. C-32, no. 2, pp. 108–120, 1983.
[33] R. A. Brooks and T. Lozano-Pérez, "A subdivision algorithm in configuration space for findpath with rotation," *IEEE Trans. Syst. Man Cybern.*, vol. SMC-15, no. 2, pp. 224–233, 1985.
[34] J. J. Kuffner and S. M. LaValle, "RRT-connect: An efficient approach to single-query path planning," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2000, pp. 995–1001.
[35] J. D. Gammell, S. S. Srinivasa, and T. D. Barfoot, "Batch informed trees (BIT*): Sampling-based optimal planning via the heuristically guided search of implicit random geometric graphs," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2015, pp. 3067–3074.
[36] J. D. Gammell, T. D. Barfoot, and S. S. Srinivasa, "Informed sampling for asymptotically optimal path planning," *IEEE Trans. Robot.*, vol. 34, no. 4, pp. 966–984, 2018.
[37] L. Janson, E. Schmerling, A. Clark, and M. Pavone, "Fast marching tree: A fast marching sampling-based method for optimal motion planning in many dimensions," *Int. J. Robot. Res.*, vol. 34, no. 7, pp. 883–921, 2015.
[38] I. A. Şucan, M. Moll, and L. E. Kavraki, "The open motion planning library," *IEEE Robot. Autom. Mag.*, vol. 19, no. 4, pp. 72–82, 2012.
[39] M. Moll, I. A. Şucan, and L. E. Kavraki, "Benchmarking motion planning algorithms: An extensible infrastructure for analysis and visualization," *IEEE Robot. Autom. Mag.*, vol. 22, no. 3, pp. 96–102, 2015.
[40] N. Ratliff, M. Zucker, J. A. Bagnell, and S. Srinivasa, "CHOMP: Gradient optimization techniques for efficient motion planning," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2009, pp. 489–494.
[41] M. Kalakrishnan, S. Chitta, E. Theodorou, P. Pastor, and S. Schaal, "STOMP: Stochastic trajectory optimization for motion planning," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2011, pp. 4569–4574.
[42] J. Schulman, Y. Duan, J. Ho, A. Lee, I. Awwal, H. Bradlow, J. Pan, S. Patil, K. Goldberg, and P. Abbeel, "Motion planning with sequential convex optimization and convex collision checking," *Int. J. Robot. Res.*, vol. 33, no. 9, pp. 1251–1270, 2014.
[43] M. Mukadam, J. Dong, X. Yan, F. Dellaert, and B. Boots, "Continuous-time gaussian process motion planning via probabilistic inference," *Int. J. Robot. Res.*, vol. 37, no. 11, pp. 1319–1340, 2018.